Sterling Fitness graph files — v0.2 (2026-10-27)

CLAUDE.md

```
# Sterling_Fitness — instructions for coding agents

This folder holds the Sterling Fitness method documents. The scoring code and
all member data live in the separate `Local LM Models` repository.

## Files

- `graph/fitness_spec.py` — the single source of truth: agents, glossary,
  exercise model, technique tables, sources, deviations, owner decisions.
- `graph/make_fitness_diagram.py` — draws the diagram from the spec.
- `graph/make_fitness_tables.py` — writes the technique tables from the spec as
  HTML in `graph/archive/<version>_<date>/`, then prints the PDF into `graph/`.
  Needs Chrome.
- `graph/*.svg`, `graph/*.pdf` and the tables HTML — generated. Never edit by hand.
- `graph/archive/` — old versions, plus the current tables HTML that
  `make_fitness_tables.py` writes. A record only; never edit by hand.

## Commands

```bash
cd Sterling_Fitness/graph
python make_fitness_diagram.py
python make_fitness_tables.py
```

Scoring tests (in `Local LM Models`): `python -m pytest tests/test_fitness_analytics.py -q`

Rules

1. Code sets every score and every pass or fail. A language model never does.
2. Log by exercise. Score the program by exercise. Check and report targets
   by body part.
3. Each exercise has a permanent id. Never match logs by name.
4. Compare weight and progress only within the same exercise.
5. Code builds the program from written rules (N3). Code checks it against the
   targets and limits, and the member approves it (G1). The member is scored on
   the program (ST1.5), never the targets. Only the assigned exercise counts.
   A program change starts on a Sunday; each score stores its program version.
6. Every target, test, norm, movement and exercise needs a published source,
   current, measured, and covering ages 50 and up. Cite it by id from `SOURCES`.
7. No member data in this folder: no results, names or test dates.
8. Use the terms in `GLOSSARY` exactly. Do not invent synonyms.

Change order

1. Change the technique table in `fitness_spec.py`.
2. Then change the code in `Local LM Models`.
3. Then regenerate the diagram and tables.

If the code and a table must differ for a while, add a `DEVIATIONS` entry.

Stop and ask the owner when

- a change needs a number or a source that is not in `SOURCES`;
- an item in `OPEN_DECISIONS` would be decided by the change;
- a change would touch the app (`mobile-app`) or the `925_Branch_refactor`
  branch. A merge there redeploys the live app.
```

fitness_spec.py

```python
"""
fitness_spec.py — Single source of truth for the Sterling Fitness agent graph
(fan-out -> reduce -> cross-check -> synthesize), modelled on the Property DD
diamond graph (Merriwood/propertydd-graph, dd_spec.py v2.8).

The diagram (make_fitness_diagram.py) is generated from the structures below, so
the picture and the spec cannot drift apart. DRAFT v0.1 for owner review: the
lanes are the owner's (2026-10-26); the agents are proposed. No technique, test
or norm is chosen here. Each agent's methods and published sources are set in
its technique table, under the evidence standard (published, current,
empirical, age-appropriate, covering ages 50 and up).

v0.2 (2026-10-27): glossary, the exercise model (body part -> movement ->
exercise) and the ST1 technique table (GLOSSARY, EXERCISE_MODEL, TECHNIQUES,
SOURCES, DEVIATIONS, OPEN_DECISIONS below), and N3 Program Builder with the
program loop S4 -> N3, approved at G1 each period. ST1 stays ONE agent: its method is the same at every
age and only the weekly target changes, so the age band is a parameter N1
supplies, not a second agent. Agents split by age only where the test itself
differs (ST2 grip at 50-59, ST3 Senior Fitness Test from 60).

Owner decisions 2026-10-27: O1, O2, O4 as recommended; O5 reversed. The member
is scored on the program (ST1.5). Targets check the program at N3 and G1, and
ST1.6 reports them, unscored.

status: "built" = runs today in fitness-analytics / the diary app;
        "data"  = the data is recorded but nothing scores it;
        "proposed" = not built.
"""

VERSION, DATE = "0.2", "2026-10-27"
ORG = "Sterling Fitness · SterlingLife"
DOC_REF = f"Agent Graph · draft v{VERSION} · {DATE} · for owner review"
CONFIDENTIAL = "Method only: no member results in this file"

TITLE = "Sterling Fitness — Wellness Agent Graph"
SUBTITLE = ("Fan-out to 7 wellness lanes  |  Reduce to one results set per lane  "
            "|  Cross-check and score in code  |  Synthesize to the diary wiki")

# ---------------------------------------------------------------------------
# Lanes. Two-letter codes, because one-letter "G" would collide with the gates.
# ---------------------------------------------------------------------------
CLUSTERS = [
    dict(key="ST", name="Strength Training", color="#4A57C4",
         reducer=("R-ST", "Strength Results Set")),
    dict(key="CA", name="Cardio Training", color="#B0454A",
         reducer=("R-CA", "Cardio Results Set")),
    dict(key="CG", name="Cognition", color="#7B4B94",
         reducer=("R-CG", "Cognition Results Set")),
    dict(key="NU", name="Nutrition", color="#2E7D52",
         reducer=("R-NU", "Nutrition Results Set")),
    dict(key="BA", name="Balance", color="#A9761C",
         reducer=("R-BA", "Balance Results Set")),
    dict(key="HV", name="Hearing / Vision", color="#2C6E8F",
         reducer=("R-HV", "Hearing & Vision Results Set")),
    dict(key="SO", name="Social Engagement", color="#8A5A3C",
         reducer=("R-SO", "Social Results Set")),
]

# ---------------------------------------------------------------------------
# Intake (the top of the diamond)
# ---------------------------------------------------------------------------
INTAKE = [
    dict(code="N0", name="Member Intake & Goals", status="data",
         mandate="Records age, sex, goals and anything that limits exercise. "
                 "Everything downstream is judged against this record.",
         output="MemberContext (age band, sex, goals, limits)"),
    dict(code="N1", name="Scope & Age-Band Profiler", status="proposed",
         mandate="Decides which agents run this quarter and which test fits the "
                 "member's age band (grip at 50-59, Senior Fitness Test from 60). "
                 "A test with no published norms for the age band does not run.",
         output="ScopePlan (agent roster, tests due, dates)"),
    dict(code="N2", name="Evidence Pipeline (service)", status="built",
         mandate="The only writer to the evidence store: app sets and weight "
```

```
                "(derive_fitness.py), quarterly test results (diary form), diary "
                "entries (masked copy for any language model).",
         output="Evidence store + masked working copy"),
    dict(code="N3", name="Program Builder", status="proposed",
         mandate="Picks the member's exercises for each movement from EXERCISE_MODEL, by "
                "written rules: limits (N0), age band (N1), rotation schedule. Code only. "
                "Starts from S4's proposal after the first period. Each body part gets "
                "from its target up to the band's limit in sets a week (upper body: push "
                "and pull each), on 2 or more days.",
         output="Program (exercises and sets per week, versioned)"),
]

# ---------------------------------------------------------------------------
# Fan-out: the lane agents (proposed roster)
# ---------------------------------------------------------------------------
NODES = [
    # Strength Training
    dict(code="ST1", cluster="ST", status="built", name="Training Volume & Muscle Balance",
         bands="50-64 · 65+",
         mandate="Weekly sets and training days per body part, against the target "
                "for the member's age band (Gate A strength)."),
    dict(code="ST2", cluster="ST", status="built", name="Grip Strength Test (50-59)",
         bands="50-59",
         mandate="Combined grip against US national norms for the age band (Gate B)."),
    dict(code="ST3", cluster="ST", status="built", name="Chair Stand & Arm Curl (60-94)",
         bands="60-94",
         mandate="Senior Fitness Test strength items against age-band norms (Gate B)."),
    dict(code="ST4", cluster="ST", status="proposed", name="Load & Progression",
         mandate="Weight lifted and effort over time. Needs load and reps-in-reserve data."),
    dict(code="ST5", cluster="ST", status="proposed", name="Muscle Loss Screen",
         mandate="Screens for age-related muscle loss (sarcopenia) from strength results."),
    # Cardio Training
    dict(code="CA1", cluster="CA", status="built", name="Aerobic Volume (MET-minutes)",
         mandate="Weekly MET-minutes against the target (Gate A cardio)."),
    dict(code="CA2", cluster="CA", status="built", name="2-Minute Step Test (60-94)",
         mandate="Senior Fitness Test endurance item against age-band norms (Gate B)."),
    dict(code="CA3", cluster="CA", status="proposed", name="Endurance Test (50-59)",
         mandate="Gap today: no endurance test is scored at 50-59."),
    dict(code="CA4", cluster="CA", status="proposed", name="Daily Steps & Activity",
         mandate="Activity outside planned training."),
    dict(code="CA5", cluster="CA", status="proposed", name="Heart Rate & Intensity",
         mandate="Effort level during cardio. Needs heart-rate data."),
    # Cognition
    dict(code="CG1", cluster="CG", status="proposed", name="Cognitive Screen",
         mandate="A validated screening test. Positive results need a qualified reviewer (G2)."),
    dict(code="CG2", cluster="CG", status="proposed", name="Mental Activity",
         mandate="Learning, reading and similar activity, from the diary."),
    dict(code="CG3", cluster="CG", status="proposed", name="Sleep",
         mandate="Sleep length and regularity."),
    dict(code="CG4", cluster="CG", status="proposed", name="Modifiable Risk Factors",
         mandate="Published modifiable risk factors. Derived health inference: §1.9 applies."),
    # Nutrition
    dict(code="NU1", cluster="NU", status="data", name="Body Weight Trend",
         mandate="Weight is recorded daily in the app; nothing scores it yet."),
    dict(code="NU2", cluster="NU", status="proposed", name="Protein Intake",
         mandate="Protein per kg of body weight. Needs a food record."),
    dict(code="NU3", cluster="NU", status="proposed", name="Diet Quality",
         mandate="A validated diet-quality score."),
    dict(code="NU4", cluster="NU", status="proposed", name="Undernutrition Screen",
         mandate="A validated screen for poor nutrition in older adults."),
    # Balance
    dict(code="BA1", cluster="BA", status="proposed", name="Single-Leg Stance",
         mandate="Static balance timed test against age-band norms."),
    dict(code="BA2", cluster="BA", status="proposed", name="8-Foot Up-and-Go",
         mandate="Senior Fitness Test agility and balance item."),
    dict(code="BA3", cluster="BA", status="proposed", name="Fall Risk Screen",
         mandate="A published fall-risk screen. A positive result is a safety flag."),
    dict(code="BA4", cluster="BA", status="proposed", name="Balance Practice (yoga, tai chi)",
         mandate="Balance and mobility sessions. Today these are recorded but unscored."),
    # Hearing / Vision
    dict(code="HV1", cluster="HV", status="proposed", name="Hearing Screen",
         mandate="A validated self-administered hearing test or questionnaire."),
    dict(code="HV2", cluster="HV", status="proposed", name="Vision Screen",
         mandate="A validated vision check or questionnaire."),
    dict(code="HV3", cluster="HV", status="proposed", name="Exams & Devices",
         mandate="Dates of last hearing and eye exams, and devices in use."),
    # Social Engagement
```

```
    dict(code="SO1", cluster="SO", status="proposed", name="Loneliness Scale",
         mandate="A validated short loneliness scale."),
    dict(code="SO2", cluster="SO", status="proposed", name="Social Network Size",
         mandate="A validated social-network scale."),
    dict(code="SO3", cluster="SO", status="proposed", name="Social Activity",
         mandate="Time with others, from the diary."),
]

# ---------------------------------------------------------------------------
# Cross-check and scoring. Two loops, both drawn: the re-test loop (X3, inside a
# period) and the program loop (S4 -> N3, once per period, so one run has no cycle).
# ---------------------------------------------------------------------------
CROSS = [
    dict(code="X0", name="Citation & Norms Verifier",
         mandate="Every result names its protocol and norm source; the member's age "
                 "is inside the norm's age band; the norms review is not overdue."),
    dict(code="X1", name="Cross-Lane Contradiction Detector",
         mandate="Finds results that disagree across lanes, such as high training "
                 "volume with falling grip, or weight loss with low protein."),
    dict(code="X2", name="Scoring Engine (code only)",
         mandate="Gate A wellness score, Gate B z-scores and Sterling Status. "
                 "Python only; no language model sets a number."),
    dict(code="X3", name="Re-Test Controller",
         mandate="A test older than 120 days, or thin evidence, sends a re-test back "
                 "to G1; the new result re-enters through N2."),
]

# ---------------------------------------------------------------------------
# Synthesis (the bottom point of the diamond)
# ---------------------------------------------------------------------------
SYNTH = [
    dict(code="S1", name="925 Score", status="built",
         mandate="Monthly Gate A score and Sterling Status."),
    dict(code="S2", name="Fitness Assessment", status="built",
         mandate="Quarterly test results against age-band norms, lane by lane."),
    dict(code="S3", name="Wiki Writer", status="built",
         mandate="fitness_wiki.py writes the diary wiki pages from the scores; "
                 "approved text blocks only."),
    dict(code="S4", name="Plan & Referrals", status="proposed",
         mandate="Next period's program, targets and tests due, and any referral a safety "
                 "flag needs. The program goes back to N3 and waits for G1."),
]

DELIVERABLE = ("DIARY WIKI  ->  Sterling Status  ·  925 Score  ·  Fitness Assessment  ·  "
               "Lane results with sources  ·  Next-quarter plan  ·  Referrals")

# ---------------------------------------------------------------------------
# Human-in-the-Loop gates
# ---------------------------------------------------------------------------
GATES = [
    dict(code="G1", name="Plan Approval Gate",
         rule="The member approves the program, roster, tests due and targets. Code first checks "
              "each body part is within target and limit, on 2+ days. Programs start on Sundays"),
    dict(code="G2", name="Screen Review Gate",
         rule="A qualified reviewer confirms any positive health screen before it is reported"),
    dict(code="G3", name="Status Gate",
         rule="Code sets Sterling Status; any safety flag goes to referral first"),
    dict(code="G3A", name="Report Sign-Off Gate",
         rule="The owner approves page layout and wording before the wiki is written"),
    dict(code="G4", name="Release Gate",
         rule="Nothing leaves the member's local store without prior written authorization (§1.9)"),
]

# ---------------------------------------------------------------------------
# Control plane (cross-cutting services, the left rail)
# ---------------------------------------------------------------------------
CONTROL = [
    dict(name="Evidence Ledger",
         note="Source record, date and protocol on every result."),
    dict(name="Norms Library",
         note="Cited norms, reviewed yearly."),
    dict(name="Capture Layer",
         note="App, test form, diary."),
    dict(name="Assumption Register",
         note="Placeholders stay labelled."),
    dict(name="Safety Flags",
         note="Positive screens go to G2."),
```

```
    dict(name="Privacy Guard",
         note="Health data stays local."),
]

OFFLINE_JOBS = ("Offline, not graph nodes: the yearly norms review, and evaluation "
                "against recorded tests.")

# ---------------------------------------------------------------------------
# Sources, by id. Every technique step names its sources by these ids.
# ---------------------------------------------------------------------------
SOURCES = {
    "WHO2020": dict(
        cite="Bull FC, Al-Ansari SS, Biddle S, et al. World Health Organization 2020 "
             "guidelines on physical activity and sedentary behaviour. Br J Sports Med "
             "2020;54(24):1451-1462. doi:10.1136/bjsports-2020-102955",
        covers="Adults 18-64 and older adults 65+, separate recommendations",
        used="Muscle-strengthening work on all major muscle groups, 2 or more days a week",
        checked="Recommendation known; re-read the text before approval"),
    "PAG2018": dict(
        cite="Piercy KL, Troiano RP, Ballard RM, et al. The Physical Activity Guidelines "
             "for Americans. JAMA 2018;320(19):2020-2028. doi:10.1001/jama.2018.14854",
        covers="US adults and older adults",
        used="Same 2-days-a-week, all-major-muscle-groups rule as WHO2020",
        checked="Recommendation known; re-read the text before approval"),
    "NSCA2019": dict(
        cite="Fragala MS, Cadore EL, Dorgo S, et al. Resistance training for older adults: "
             "position statement from the National Strength and Conditioning Association. "
             "J Strength Cond Res 2019;33(8):2019-2052. doi:10.1519/JSC.0000000000003230",
        covers="Older adults (the NSCA's first statement for this group)",
        used="2-3 sets per major muscle group, 2-3 sessions a week",
        checked="Citation confirmed (PubMed 31343601). Set and session figures confirmed "
                "only through a published summary; read the full text before approval"),
    "BORDE2015": dict(
        cite="Borde R, Hortobágyi T, Granacher U. Dose-response relationships of resistance "
             "training in healthy old adults: a systematic review and meta-analysis. "
             "Sports Med 2015;45(12):1693-1720. doi:10.1007/s40279-015-0385-9",
        covers="25 randomized trials, mean age 65 and over",
        used="2-3 sets per exercise; 3 sessions a week",
        checked="Citation and inclusion rule confirmed. Dose figures from secondary "
                "sources only, and they disagree on intensity (50-70% vs 70-79% of 1RM), "
                "so intensity is not used. Read the full text (PMC4656698) before approval"),
    "SCHOEN2017": dict(
        cite="Schoenfeld BJ, Ogborn D, Krieger JW. Dose-response relationship between weekly "
             "resistance training volume and increases in muscle mass: a systematic review "
             "and meta-analysis. J Sports Sci 2017;35(11):1073-1082. "
             "doi:10.1080/02640414.2016.1210197",
        covers="Mostly young adults: NOT age-appropriate under the evidence standard",
        used="Context only: sets per muscle group per week is the standard volume unit; "
             "more weekly sets gave more growth up to 10+",
        checked="Findings confirmed through the authors' published comments"),
    "RALSTON2017": dict(
        cite="Ralston GW, Kilgore L, Wyatt FB, Baker JS. The effect of weekly set volume on "
             "strength gain: a meta-analysis. Sports Med 2017;47(12):2585-2601. "
             "doi:10.1007/s40279-017-0762-7",
        covers="Mostly young adults: NOT age-appropriate under the evidence standard",
        used="Context only: graded dose-response between weekly sets and strength",
        checked="Citation and finding confirmed (PubMed 28755103)"),
}

# ---------------------------------------------------------------------------
# Glossary and code key: every term is defined once here and used the same
# way in every document. The tables page prints both at the top.
# ---------------------------------------------------------------------------
GLOSSARY = [
    ("Body part", "body_part", "What gets scored, e.g. Legs."),
    ("Movement", "movement", "The kind of motion, e.g. Knee bend."),
    ("Main muscles", "muscles", "The muscles a movement works. Described, never scored."),
    ("Exercise", "exercise_id", "What the member does and logs, e.g. Leg Press (legPress). The id never changes; the name
may."),
    ("Set", "sets", "One round of an exercise, as logged in the app."),
    ("Age band", "age_band", "The member's age range. It picks targets and tests."),
    ("Target", "target", "The weekly sets a body part should get. Same for everyone in "
                         "an age band. Every program must meet it."),
    ("Limit", "limit", "The most weekly sets a program may assign a body part. Same for "
                       "everyone in an age band."),
    ("Program", "program", "The exercises and weekly sets assigned to one member for a "
                           "period. Built by code (N3), approved by the member (G1). Each "
```

```
                            "version starts on a Sunday."),
    ("Program score", "program_score", "How much of the week's program the member did. "
                                       "The member's strength score."),
]

CODE_KEY = [
    ("N0-N3", "Intake: member record, age-band planner, evidence store, program builder"),
    ("ST1-ST5 ...", "Agents, by lane (ST = Strength, CA = Cardio, ...)"),
    ("R-xx", "Results set for one lane"),
    ("X0-X3", "Checks: sources, contradictions, scoring, re-tests"),
    ("S1-S4", "Outputs: 925 score, assessment, wiki, plan"),
    ("G1-G4", "Approval gates: a person must approve before work continues"),
    ("D# / O#", "Deviation (code differs from this table) / open owner decision"),
]

# ---------------------------------------------------------------------------
# Exercise model (proposed, documents only). Body part -> Movement -> Exercise.
# Three body parts. The scoring code still uses four groups (push and pull
# separate): see D5. Exercise ids are the app's ids (exercises.ts). Muscles
# describe a movement; they are not scored. Every row needs a source (rule 6).
# ---------------------------------------------------------------------------
BODY_PARTS = [
    ("upper_body", "Upper body"),
    ("core", "Core"),
    ("legs", "Legs"),
]

# One dict per movement. side: "push" or "pull" (upper body only, see O4).
# exercises: (exercise_id, name, where, note). where: "app" = in the app today,
# "logged" = logged but skipped by the code (D4), "new" = not in the app yet.
# The app's own ids stay as they are (rule 2); new ids use the same camelCase.
EXERCISE_MODEL = [
    # Upper body
    dict(part="upper_body", side="push", movement="Push forward",
         muscles="Chest, front shoulders, triceps",
         exercises=[("chestPress", "Chest Press", "app", ""),
                    ("pushUp", "Push-up", "logged", ""),
                    ("wallPushUp", "Wall Push-up", "new", "")]),
    dict(part="upper_body", side="push", movement="Push overhead",
         muscles="Shoulders, triceps",
         exercises=[("military", "Overhead Press", "app", "App name today: Military"),
                    ("seatedDumbbellPress", "Seated Dumbbell Shoulder Press", "new", "")]),
    dict(part="upper_body", side="push", movement="Arm straighten",
         muscles="Triceps",
         exercises=[("triceps", "Triceps Extension", "app", "App name today: Triceps"),
                    ("tricepsPushdown", "Triceps Pushdown", "new", "")]),
    dict(part="upper_body", side="pull", movement="Pull down",
         muscles="Upper back (lats), biceps",
         exercises=[("lat", "Lat Pulldown", "app", "App name today: Lat"),
                    ("overheadPull", "Overhead Pull", "app", "Works the lats (O3)"),
                    ("assistedPullUp", "Assisted Pull-up", "new", "")]),
    dict(part="upper_body", side="pull", movement="Pull back",
         muscles="Middle back, rear shoulders, biceps",
         exercises=[("row", "Seated Row", "app", "App name today: Row"),
                    ("bandRow", "Band Row", "new", "")]),
    dict(part="upper_body", side="pull", movement="Arm bend",
         muscles="Biceps",
         exercises=[("bicepCurls", "Biceps Curl", "app", ""),
                    ("hammerCurl", "Hammer Curl", "new", "")]),
    dict(part="upper_body", side="pull", movement="Shoulder lift",
         muscles="Upper back (upper trapezius)",
         exercises=[("crunch", "Shoulder Crunch", "app", "App name today: Crunch. Also "
                                                         "called a dumbbell shrug. Not a "
                                                         "stomach crunch. O3")]),
    # Core
    dict(part="core", side="", movement="Trunk curl",
         muscles="Front stomach muscles (rectus abdominis)",
         exercises=[("abs", "Ab Crunch", "app", "App name today: Abs"),
                    ("abMachineCrunch", "Ab Machine Crunch", "new", "")]),
    dict(part="core", side="", movement="Trunk twist and side bend",
         muscles="Side stomach muscles (obliques)",
         exercises=[("sideBend", "Side Bend", "new", ""),
                    ("standingCableTwist", "Standing Cable Twist", "new", "")]),
    dict(part="core", side="", movement="Back raise",
         muscles="Lower back",
         exercises=[("birdDog", "Bird Dog", "new", ""),
                    ("backExtension", "Back Extension", "new", "")]),
```

```
    dict(part="core", side="", movement="Hold steady",
         muscles="Deep stomach muscles, lower back",
         exercises=[("plank", "Plank", "new", ""),
                    ("wallPlank", "Wall Plank", "new", "")]),
    # Legs
    dict(part="legs", side="", movement="Knee bend",
         muscles="Quadriceps, glutes",
         exercises=[("legPress", "Leg Press", "app", ""),
                    ("squat", "Squat", "logged", ""),
                    ("sitToStand", "Sit-to-Stand", "new", "")]),
    dict(part="legs", side="", movement="Hip bend",
         muscles="Glutes, hamstrings",
         exercises=[("hipBridge", "Hip Bridge", "new", ""),
                    ("hipHinge", "Hip Hinge", "new", ""),
                    ("deadlift", "Deadlift", "new", "")]),
    dict(part="legs", side="", movement="Knee curl",
         muscles="Hamstrings",
         exercises=[("hamstringCurl", "Hamstring Curl", "new", "")]),
    dict(part="legs", side="", movement="Leg out to the side",
         muscles="Hip abductors (glute medius, tensor fasciae latae)",
         exercises=[("sideLegRaise", "Side Leg Raise", "new", ""),
                    ("hipAbductionMachine", "Hip Abduction Machine", "new", "")]),
    dict(part="legs", side="", movement="Leg in",
         muscles="Adductors (inner thigh)",
         exercises=[("hipAdductionMachine", "Hip Adduction Machine", "new", ""),
                    ("ballSqueeze", "Ball Squeeze", "new", "")]),
    dict(part="legs", side="", movement="Knee lift",
         muscles="Hip flexors",
         exercises=[("standingKneeRaise", "Standing Knee Raise", "new", ""),
                    ("marchInPlace", "March in Place", "new", "")]),
    dict(part="legs", side="", movement="Heel raise",
         muscles="Calves",
         exercises=[("heelRaise", "Heel Raise", "new", "")]),
    dict(part="legs", side="", movement="Toe raise",
         muscles="Shin (tibialis anterior)",
         exercises=[("toeRaise", "Toe Raise", "new", "")]),
]

EXERCISE_RULES = [
    "Log by exercise. Score the program by exercise. Check and report targets by body part.",
    "Each exercise has a permanent id. Never match logs by name.",
    "Compare weight and progress only within the same exercise (ST4).",
    "Code builds the program from written rules (N3). A language model never does.",
    "The member approves the program before it starts (G1).",
    "Score against the program. Every program meets the targets, so a member who "
    "falls short of the program scores lower.",
    "Only the assigned exercise counts toward the program score. A substitute is "
    "logged and counts toward the target report, but earns no program score.",
    "A program change starts on a Sunday. Each week is scored against the program "
    "version in force that week, and the score stores that version.",
    "Every movement and exercise needs a published source before it ships.",
]

# ---------------------------------------------------------------------------
# Technique tables (FIT-T). One entry per agent. Four columns per step:
# step, what it does, rule or source, status. Longer notes go to an appendix.
# An empty source list means a program rule set by the owner, not research.
# ---------------------------------------------------------------------------
TECHNIQUES = {
    "ST1": dict(
        purpose="Scores how much of the week's program the member did, and reports "
                "whether each body part got enough strength work, often enough, for the "
                "member's age band. One agent for all ages: only the target changes, and "
                "N1 supplies the band.",
        steps=[
            dict(id="ST1.1", step="Load program and target",
                 does="Loads the program version in force this week (approved at G1, "
                      "started on a Sunday), and the targets for the age band from N1. Stops "
                      "and reports if there is no approved program or target.",
                 sources=["NSCA2019", "BORDE2015"], status="proposed",
                 note="First, because the band is known before ST1 runs. Range: 2-3 sets "
                      "per body part per session, 2-3 sessions a week, so 4-9 sets a week. "
                      "Target 6 sets a week per body part; upper body counts push 6 and "
                      "pull 6. Limit 9. Ages 50-64 use the 65+ range, labelled 'not "
                      "validated under 65'. The code still uses a placeholder table. See "
                      "D3, O1, O2, O4."),
            dict(id="ST1.2", step="Find body part",
```

```
                does="Looks up the body part of each logged exercise in the exercise model.",
                sources=[], status="built",
                note="The guidelines (WHO2020, PAG2018) ask for all major muscle groups; "
                     "the three body parts are the program's own split. See D4, D5."),
            dict(id="ST1.3", step="Count sets",
                does="Adds up sets per exercise and per body part, Sunday to Saturday.",
                sources=[], status="built",
                note="Every logged set counts, substitutes included. See D1."),
            dict(id="ST1.4", step="Count days",
                does="Counts the days each body part was trained. Rule: 2 or more.",
                sources=["WHO2020", "PAG2018"], status="proposed",
                note="Same rule for every age band. How a body part trained on one day "
                     "affects the program score: see O6. Not counted today: D2."),
            dict(id="ST1.5", step="Program score",
                does="Per assigned exercise: sets done of that exercise / sets assigned, "
                     "capped at 1. Program score = 100 x the average over the assigned "
                     "exercises. The member's strength score.",
                sources=[], status="proposed",
                note="Matched by exercise id, so a substitute earns nothing here. The cap "
                     "stops extra sets of one exercise from making up for one that was "
                     "skipped, and gives no reason to overtrain. Sets only: the app records "
                     "no weight or effort (ST4). See O5, D7."),
            dict(id="ST1.6", step="Target report",
                does="Per body part: sets / target, capped at 1 (upper body: push and pull "
                     "each). Reported, not scored.",
                sources=["NSCA2019", "BORDE2015"], status="built",
                note="Built today as the score itself (D7). Shows whether the body got "
                     "enough work whatever the program said."),
            dict(id="ST1.7", step="Report",
                does="Sends the program score, the program version, sets, days, age band, "
                     "the target report and the target's sources to R-ST.",
                sources=[], status="proposed",
                note="X0 needs the sources to check every result. The program version "
                     "keeps old scores fixed when the program changes."),
        ]),
}

DEVIATIONS = [
    dict(id="D1", agent="ST1",
         text="The fitness_analytics.py docstring describes a 'hard set' rule (3 or fewer "
              "reps in reserve, or at least 60% of estimated 1RM). The code counts every "
              "logged set, because the app records no weight or effort. Fix: reword the "
              "docstring to match the code. Weight and effort belong to ST4."),
    dict(id="D2", agent="ST1",
         text="Training days per body part (ST1.4) are not counted or scored."),
    dict(id="D3", agent="ST1",
         text="The weekly targets in the code are one placeholder table for every age (legs "
              "10, push 8, pull 8, core 5), with no source. Decided: 6 a week per body part, "
              "push and pull 6 each, limit 9 (O1, O2, O4)."),
    dict(id="D4", agent="ST1",
         text="Squats and push-ups are logged but skipped by the code (EXCLUDED_EXERCISES). "
              "The exercise model counts them."),
    dict(id="D5", agent="ST1",
         text="The scoring code uses four groups (lower_body, upper_push, upper_pull, core) "
              "and averages four. The model has three body parts (legs, upper_body, core)."),
    dict(id="D6", agent="ST1",
         text="The scoring code matches logs by the app's names (e.g. 'Military'), not by id. "
              "Switch matching to ids before any app name changes, or logged sets stop counting."),
    dict(id="D7", agent="ST1",
         text="The scoring code scores sets against targets (ST1.6). The method scores the "
              "program (ST1.5). Until the code changes, the strength score is the target "
              "score, and no program is stored."),
]

# status: "open", or the date the owner decided it.
OPEN_DECISIONS = [
    dict(id="O1", agent="ST1", status="decided 2026-10-27",
         text="Ages 50-64: no study on weekly sets was found for this age group. Decided: "
              "use the 65+ range (4-9) and label it 'not validated under 65'. It is the "
              "lower, safer dose."),
    dict(id="O2", agent="ST1", status="decided 2026-10-27",
         text="The weekly target. Decided: 6 sets per body part, e.g. 3 sets on 2 days or 2 "
              "sets on 3 days. Limit: 9, the top of the range."),
    dict(id="O3", agent="ST1", status="decided 2026-10-27",
         text="Which movement are Overhead Pull and Crunch (shoulder crunch)? Decided: both "
              "are upper body, pull. Overhead Pull works the lats (Pull down). Shoulder "
              "Crunch (a dumbbell shrug) works the upper back, upper trapezius (Shoulder lift)."),
```

```
    dict(id="O4", agent="ST1", status="decided 2026-10-27",
         text="Upper body holds both push and pull. Decided: the upper body target counts "
              "push and pull separately (6 sets each), using each movement's side."),
    dict(id="O5", agent="ST1", status="decided 2026-10-27",
         text="Does following the program count toward the score? Decided: yes, it is the "
              "only score (ST1.5), and future points build on it. Assigned exercises are "
              "program components; a substitute earns nothing. N3 builds every program at "
              "or above target, so falling short of the program scores lower. Program "
              "changes start on Sunday. The program is weekly sets per exercise, with "
              "each body part on 2 or more days."),
    dict(id="O6", agent="ST1", status="open",
         text="A body part trained on only one day in a week. Recommended: its exercises "
              "earn at most half their program credit that week, so the 2-day rule counts "
              "without a separate score."),
]
```

make_fitness_diagram.py

```python
"""
make_fitness_diagram.py — renders the agent graph in fitness_spec.py to one SVG.
Adapted from make_diagram.py in Merriwood/propertydd-graph v2.8. Layout is
computed from the spec, not hand-placed: adding an agent or a lane re-flows the
picture. Never edit the SVG by hand; change the spec and run this again.
"""
import textwrap
from fitness_spec import (VERSION, DATE, ORG, DOC_REF, CONFIDENTIAL, TITLE, SUBTITLE,
                      CLUSTERS, INTAKE, NODES, CROSS, SYNTH, DELIVERABLE, GATES,
                      CONTROL, OFFLINE_JOBS, TECHNIQUES)

# ---------------------------------------------------------------- palette
INK, MUTED, LINE, GATE = "#1B2430", "#5A6672", "#C3CBD4", "#B5761B"
BG, PANEL = "#FFFFFF", "#F5F7F9"

# ---------------------------------------------------------------- layout
W = 1500
RAIL_X, RAIL_W = 16, 96
COL_X0, GAP, RIGHT = 132, 12, 72   # right margin holds the re-test and program loops
NCOL = len(CLUSTERS)
COL_W = int((W - COL_X0 - RIGHT - (NCOL - 1) * GAP) / NCOL)
BANDW = W - COL_X0 - RIGHT
CW = int((BANDW - (len(CROSS) - 1) * GAP) / len(CROSS))   # cross-check box width
X_IN = COL_X0 + CW / 2                                    # every lane enters X0 first
MAXROWS = max(len([n for n in NODES if n["cluster"] == c["key"]]) for c in CLUSTERS)

TOP_Y, HDR_H, NH, NGAP = 208, 24, 40, 5
NODES_Y = TOP_Y + HDR_H + 8
FAN_BOTTOM = NODES_Y + (MAXROWS - 1) * (NH + NGAP) + NH
G2_Y = FAN_BOTTOM + 8
RED_Y = G2_Y + 52
CROSS_Y = RED_Y + 86
G3_Y = CROSS_Y + 56
SYN_Y = G3_Y + 34
G3A_Y = SYN_Y + 52                 # G3A sits under S3 only
G4_Y = G3A_Y + 28
DEL_Y = G4_Y + 32
H = DEL_Y + 124

esc = lambda s: s.replace("&", "&amp;").replace("<", "&lt;").replace(">", "&gt;")


def wrap(s, n):
    return textwrap.wrap(s, n) or [""]


def box(x, y, w, h, fill, stroke, rx=6, sw=1.1, dash=None, op=1.0):
    d = f' stroke-dasharray="{dash}"' if dash else ""
    return (f'<rect x="{x}" y="{y}" width="{w}" height="{h}" rx="{rx}" fill="{fill}" '
            f'fill-opacity="{op}" stroke="{stroke}" stroke-width="{sw}"{d}/>')


def text(x, y, s, size=12, fill=INK, weight="normal", anchor="start", ls=0):
    return (f'<text x="{x}" y="{y}" font-family="Helvetica,Arial,sans-serif" '
            f'font-size="{size}" fill="{fill}" font-weight="{weight}" '
            f'text-anchor="{anchor}" letter-spacing="{ls}">{esc(s)}</text>')


TAG = {"built": "BUILT", "data": "DATA ONLY", "proposed": ""}


def node_box(x, y, w, h, code, name, color, status="proposed", bands=""):
    proposed = status == "proposed"
    o = [box(x, y, w, h, color if not proposed else BG, color, rx=6, sw=1.1,
             dash="4 3" if proposed else None, op=0.10 if not proposed else 1.0),
         f'<rect x="{x}" y="{y}" width="3.5" height="{h}" rx="1.5" fill="{color}"/>',
         text(x + 9, y + 14, code, 9.2, color, "bold", ls=0.4)]
    # Age bands and the technique-table marker share the code line.
    meta = " · ".join(t for t in (bands, "FIT-T" if code in TECHNIQUES else "") if t)
    if meta:
        o.append(text(x + 9 + 7 * len(code), y + 14, meta, 7.6, MUTED, "bold"))
    if TAG[status]:
        o.append(text(x + w - 6, y + 14, TAG[status], 7.4, color, "bold", "end", ls=0.5))
```

```
    for i, ln in enumerate(wrap(name, int((w - 16) / 6.5))[:2]):
        o.append(text(x + 9, y + 26 + i * 11, ln, 11, INK, "bold"))
    return "".join(o)


def gate_bar(x, y, w, g, h=20, label=None):
    o = [box(x, y, w, h, GATE, GATE, rx=5, sw=1.0, dash="5 3", op=0.09)]
    cx, cy = x + 14, y + h / 2
    o.append(f'<path d="M{cx} {cy-6} L{cx+6} {cy} L{cx} {cy+6} L{cx-6} {cy} Z" fill="{GATE}"/>')
    label = label or f"{g['code']}  {g['name']}  —  {g['rule']}"
    cap = int((w - 44) / 5.25)
    if len(label) > cap:
        label = label[:cap].rsplit(" ", 1)[0] + " ..."
    o.append(text(x + 27, y + h / 2 + 4, label, 9.6, "#6B4A11", "bold"))
    return "".join(o)


def shorten(t, n):
    return t if len(t) <= n else t[:n].rsplit(" ", 1)[0] + "..."


def band_label(x, y, s):
    return text(x, y, s, 10.5, MUTED, "bold", ls=1.6)


def build():
    gate = {g["code"]: g for g in GATES}
    s = [f'<svg viewBox="0 0 {W} {H}" xmlns="http://www.w3.org/2000/svg">',
         f'<rect width="{W}" height="{H}" fill="{BG}"/>',
         text(RAIL_X, 30, TITLE, 19, INK, "bold"),
         text(RAIL_X, 48, SUBTITLE, 10.5, MUTED),
         # ---- title block (right) ----
         f'<line x1="{W-RIGHT-300}" y1="12" x2="{W-RIGHT-300}" y2="52" stroke="{LINE}" '
         f'stroke-width="1"/>',
         f'<rect x="{W-RIGHT-284}" y="17" width="9" height="9" rx="2" fill="{INK}"/>',
         text(W - RIGHT - 268, 26, ORG, 12.5, INK, "bold"),
         text(W - RIGHT - 268, 40, DOC_REF, 8.6, MUTED),
         text(W - RIGHT - 268, 51, CONFIDENTIAL, 8.6, MUTED, "italic"),
         f'<line x1="{RAIL_X}" y1="58" x2="{W-RIGHT}" y2="58" stroke="{LINE}"/>']

    # ---- control rail ----------------------------------------------------
    s.append(box(RAIL_X, 72, RAIL_W, FAN_BOTTOM - 72, PANEL, LINE, rx=8))
    s.append(text(RAIL_X + 8, 90, "CONTROL PLANE", 8.4, MUTED, "bold", ls=0.9))
    yy = 104
    for c in CONTROL:
        nl = wrap(c["name"], 15)
        for i, ln in enumerate(nl):
            s.append(text(RAIL_X + 8, yy + i * 11, ln, 9.8, INK, "bold"))
        yy += 11 * len(nl) + 3
        for ln in wrap(shorten(c["note"], 92), 18)[:6]:
            s.append(text(RAIL_X + 8, yy + 9, ln, 8.1, MUTED))
            yy += 9.4
        yy += 10
    s.append(text(RAIL_X + 8, FAN_BOTTOM - 12, "runs beside every node", 8.1, MUTED, "italic"))

    # ---- band 1: intake --------------------------------------------------
    s.append(band_label(COL_X0, 84, "1 · INTAKE & SCOPING"))
    iw = int((BANDW - (len(INTAKE) - 1) * GAP) / len(INTAKE))
    for i, n in enumerate(INTAKE):
        x = COL_X0 + i * (iw + GAP)
        s.append(box(x, 92, iw, 64, "#E9ECF5", "#8B93B8", rx=7))
        s.append(text(x + 11, 109, n["code"], 9.5, "#4A57C4", "bold", ls=0.4))
        for j, ln in enumerate(wrap(n["name"], 32)[:2]):
            s.append(text(x + 11, 123 + j * 12.5, ln, 12, INK, "bold"))
        s.append(text(x + 11, 150, wrap(n["output"], 54)[0], 8.6, MUTED))
    s.append(gate_bar(COL_X0, 164, BANDW, gate["G1"]))

    # ---- band 2: fan-out -------------------------------------------------
    s.append(band_label(COL_X0, 200, "2 · FAN-OUT — PARALLEL LANE AGENTS"))
    s.append(text(W - RIGHT, 200,
                  f"{len(NODES)} agents ({sum(n['status'] == 'built' for n in NODES)} built) · roster by age band · isolated context  |  solid = built or recorded · dashed = proposed · FIT-T = technique table written",
                  9.4, MUTED, "italic", anchor="end"))
    hub = (COL_X0 + BANDW / 2, 184)
    for ci, c in enumerate(CLUSTERS):
        x = COL_X0 + ci * (COL_W + GAP)
        s.append(f'<path d="M{hub[0]} {hub[1]+4} C {hub[0]} {TOP_Y-6}, {x+COL_W/2} {TOP_Y-14}, '
```

```
                f'{x+COL_W/2} {TOP_Y}" fill="none" stroke="{c["color"]}" stroke-width="0.9" '
                f'opacity="0.5"/>')
        s.append(box(x, TOP_Y, COL_W, HDR_H, c["color"], c["color"], rx=5, op=0.92))
        s.append(text(x + COL_W / 2, TOP_Y + 16, f'{c["key"]} · {c["name"]}', 10.0,
                      "#FFFFFF", "bold", anchor="middle"))
        lane = [n for n in NODES if n["cluster"] == c["key"]]
        for ni, n in enumerate(lane):
            y = NODES_Y + ni * (NH + NGAP)
            s.append(node_box(x, y, COL_W, NH, n["code"], n["name"], c["color"], n["status"],
                              n.get("bands", "")))
        last = NODES_Y + (len(lane) - 1) * (NH + NGAP) + NH
        s.append(f'<line x1="{x+COL_W/2}" y1="{last}" x2="{x+COL_W/2}" y2="{G2_Y-4}" '
                 f'stroke="{c["color"]}" stroke-width="0.9" stroke-dasharray="3 3" '
                 f'opacity="0.55"/>')
    s.append(gate_bar(COL_X0, G2_Y, BANDW, gate["G2"]))

    # ---- band 3: reduce ----------------------------------------------------
    s.append(band_label(RAIL_X, RED_Y - 8, "3 · REDUCE — ONE RESULTS SET PER LANE"))
    for ci, c in enumerate(CLUSTERS):
        x = COL_X0 + ci * (COL_W + GAP)
        s.append(box(x, RED_Y, COL_W, 46, c["color"], c["color"], rx=6, sw=1.6, op=0.16))
        s.append(text(x + 9, RED_Y + 14, c["reducer"][0], 9.2, c["color"], "bold", ls=0.4))
        for j, ln in enumerate(wrap(c["reducer"][1], int((COL_W - 16) / 6.2))[:2]):
            s.append(text(x + 9, RED_Y + 27 + j * 11, ln, 10.6, INK, "bold"))
        s.append(f'<path d="M{x+COL_W/2} {RED_Y+46} C {x+COL_W/2} {RED_Y+62}, {X_IN} {RED_Y+62}, '
                 f'{X_IN} {CROSS_Y}" fill="none" stroke="{c["color"]}" stroke-width="1" '
                 f'opacity="0.55"/>')
    for i, t in enumerate(["merge · rank", "trend", "propose only"]):
        s.append(text(RAIL_X, RED_Y + 18 + i * 12, t, 9, MUTED, "bold"))

    # ---- band 4: cross-check -----------------------------------------------
    s.append(band_label(RAIL_X, CROSS_Y - 8, "4 · CROSS-CHECK"))
    for i, n in enumerate(CROSS):
        x = COL_X0 + i * (CW + GAP)
        s.append(box(x, CROSS_Y, CW, 48, "#FDF6E7", GATE, rx=6, sw=1.2))
        s.append(text(x + 11, CROSS_Y + 17, n["code"], 9.5, GATE, "bold", ls=0.4))
        for j, ln in enumerate(wrap(n["name"], int((CW - 22) / 6.8))[:2]):
            s.append(text(x + 11, CROSS_Y + 31 + j * 11.5, ln, 11.5, INK, "bold"))
        if i:                                          # X0 -> X1 -> X2 -> X3
            ax = x - GAP / 2
            s.append(f'<path d="M{ax-4} {CROSS_Y+20} l 8 4 l -8 4 Z" fill="{GATE}"/>')
    RE = W - RIGHT
    s.append(f'<path d="M{RE} {CROSS_Y+24} C {W-34} {CROSS_Y+24}, {W-34} {TOP_Y+8}, '
             f'{RE} {TOP_Y+8}" fill="none" stroke="{GATE}" stroke-width="1.2" '
             f'stroke-dasharray="6 4"/>')
    s.append(f'<path d="M{RE} {TOP_Y+8} l 9 -4 l 0 8 Z" fill="{GATE}"/>')
    mid = (TOP_Y + CROSS_Y) / 2
    s.append(f'<text x="{W-36}" y="{mid}" font-family="Helvetica,Arial,sans-serif" font-size="9" '
             f'fill="{GATE}" font-weight="bold" text-anchor="middle" '
             f'transform="rotate(90 {W-36} {mid})">CONDITIONAL RE-TEST — '
             f'old test or thin evidence</text>')
    # ---- evidence return path: everything durable re-enters through N2 ----
    EV = "#2C6E8F"
    ch = COL_X0 - 12
    s.append(f'<path d="M{COL_X0} {CROSS_Y+40} C {ch} {CROSS_Y+40}, {ch} {CROSS_Y}, {ch} 150 '
             f'C {ch} 130, {ch+4} 124, {COL_X0-2} 124" fill="none" stroke="{EV}" '
             f'stroke-width="1.1" stroke-dasharray="5 4" opacity="0.85"/>')
    s.append(f'<path d="M{COL_X0-2} 124 l -8 -4 l 0 8 Z" fill="{EV}" transform="translate(10,0)"/>')
    s.append(f'<text x="{ch-7}" y="{(CROSS_Y+150)/2}" font-family="Helvetica,Arial,sans-serif" '
             f'font-size="8.2" fill="{EV}" font-weight="bold" text-anchor="middle" '
             f'transform="rotate(-90 {ch-7} {(CROSS_Y+150)/2})">NEW TEST RESULTS -> N2</text>')
    s.append(gate_bar(COL_X0, G3_Y, BANDW, gate["G3"]))

    # ---- band 5: synthesize ------------------------------------------------
    s.append(band_label(RAIL_X, SYN_Y - 8, "5 · SYNTHESIZE"))
    sw_ = int((BANDW - 3 * GAP) / 4)
    for i, n in enumerate(SYNTH):
        x = COL_X0 + i * (sw_ + GAP)
        s.append(box(x, SYN_Y, sw_, 46, "#EAF1EC", "#2E7D52", rx=6, sw=1.3))
        s.append(text(x + 11, SYN_Y + 17, n["code"], 9.5, "#2E7D52", "bold", ls=0.4))
        for j, ln in enumerate(wrap(n["name"], 32)[:2]):
            s.append(text(x + 11, SYN_Y + 30 + j * 11, ln, 11, INK, "bold"))
        if n["code"] == "S3":                          # S3 -> G3A -> G4
            s.append(f'<line x1="{x+sw_/2}" y1="{SYN_Y+46}" x2="{x+sw_/2}" y2="{G3A_Y}" '
                     f'stroke="{GATE}" stroke-width="1"/>')
            s.append(gate_bar(x, G3A_Y, sw_, gate["G3A"],
                              label="G3A  Report Sign-Off — owner approves the pages"))
```

```
    s.append(gate_bar(COL_X0, G4_Y, BANDW, gate["G4"]))

    # ---- program loop: S4 proposes next period's program, back to N3 -----
    PG, LX, RE = "#2E7D52", W - 10, W - RIGHT
    s.append(f'<path d="M{RE} {SYN_Y+23} H {LX} V 124 H {RE+8}" fill="none" stroke="{PG}" '
             f'stroke-width="1.2" stroke-dasharray="6 4"/>')
    s.append(f'<path d="M{RE} 124 l 9 -4 l 0 8 Z" fill="{PG}"/>')
    pmid = (124 + SYN_Y) / 2
    s.append(f'<text x="{LX-15}" y="{pmid}" font-family="Helvetica,Arial,sans-serif" font-size="9" '
             f'fill="{PG}" font-weight="bold" text-anchor="middle" '
             f'transform="rotate(90 {LX-15} {pmid})">NEXT PROGRAM — S4 -&gt; N3</text>')

    # ---- deliverable ------------------------------------------------------
    s.append(box(RAIL_X, DEL_Y, W - RIGHT - RAIL_X + 26, 40, INK, INK, rx=7, op=0.94))
    s.append(text((W - RIGHT + 26 + RAIL_X) / 2, DEL_Y + 25, DELIVERABLE, 11.4,
                  "#FFFFFF", "bold", anchor="middle"))
    s.append(text(RAIL_X, DEL_Y + 58,
                  "Diamond invariant: agents never read each other's raw output — they contribute "
                  "typed, cited results to a ledger; only the reducers and the cross-check layer "
                  "compare across lanes.", 9.2, MUTED, "italic"))
    s.append(text(RAIL_X, DEL_Y + 70,
                  "Scoring invariant: code, never a language model, sets every score, z-score and "
                  "pass or fail. The wiki is output only: it is rebuilt from the evidence and "
                  "never read back as evidence.", 9.2, MUTED, "italic"))
    s.append(text(RAIL_X, DEL_Y + 82,
                  "Loops: the re-test loop runs inside a period; the program loop runs once per "
                  "period, so a single run has no cycle. The member is scored on the program; "
                  "G1 checks every program against the targets first.",
                  9.2, MUTED, "italic"))
    s.append(text(RAIL_X, DEL_Y + 94, OFFLINE_JOBS, 9.2, MUTED, "italic"))
    s.append("</svg>")
    return "".join(s)


OUT = f"{DATE.replace('-', '')}_sterling-fitness-agent-graph_v{VERSION}.svg"

if __name__ == "__main__":
    open(OUT, "w", encoding="utf-8").write(build())
    print(f"{OUT} written — {W}x{H}, {len(NODES)} agents, {NCOL} lanes")
```

make_fitness_tables.py

```python
"""
make_fitness_tables.py — renders the glossary, the exercise model and the
technique tables (FIT-T) in fitness_spec.py to one HTML page, the companion to
make_fitness_diagram.py. Generated, never edited by hand: change the spec and
run this again.

The HTML page goes to archive/v<VERSION>_<DATE>/. It is printed with Chrome to
a PDF in this folder, which is the copy to read.
"""
import html
import pathlib
import subprocess
from fitness_spec import (VERSION, DATE, ORG, CONFIDENTIAL, NODES, GLOSSARY, CODE_KEY,
                          BODY_PARTS, EXERCISE_MODEL, EXERCISE_RULES, TECHNIQUES, SOURCES,
                          DEVIATIONS, OPEN_DECISIONS)

esc = html.escape
NODE = {n["code"]: n for n in NODES}
TAG = {"app": "· in app", "logged": "· logged, skipped (D4)", "new": ""}

CSS = """
@page { size: letter landscape; margin: 0.5in; }
body { font-family: Helvetica, Arial, sans-serif; color: #1B2430; margin: 0; font-size: 10.5pt; }
h1 { font-size: 17pt; margin: 0 0 2pt; }
h2 { font-size: 13pt; margin: 16pt 0 4pt; border-bottom: 1px solid #C3CBD4; padding-bottom: 3pt; }
.meta { color: #5A6672; font-size: 9pt; margin-bottom: 8pt; }
table { border-collapse: collapse; width: 100%; margin: 6pt 0; }
th, td { border: 1px solid #C3CBD4; padding: 4pt 5pt; vertical-align: top; text-align: left; }
th { background: #F5F7F9; font-size: 9pt; }
td { font-size: 9.2pt; }
.built { color: #2E7D52; font-weight: bold; }
.proposed { color: #B5761B; font-weight: bold; }
.gap { color: #B0454A; font-weight: bold; }
p, li { margin: 3pt 0; }
"""


def table(head, rows):
    o = ["<table><tr>" + "".join(f"<th>{h}</th>" for h in head) + "</tr>"]
    for r in rows:
        o.append("<tr>" + "".join(f"<td>{c}</td>" for c in r) + "</tr>")
    return "".join(o) + "</table>"


def build():
    o = [f"<!doctype html><html><head><meta charset='utf-8'><title>FIT-T v{VERSION}</title>"
         f"<style>{CSS}</style></head><body>",
         "<h1>Sterling Fitness — Technique Tables (FIT-T)</h1>",
         f"<div class='meta'>{esc(ORG)} · draft v{VERSION} · {DATE} · for owner review · "
         f"{esc(CONFIDENTIAL)}. Generated from fitness_spec.py.</div>",
         "<h2>Terms</h2>",
         table(["Term", "Code name", "Meaning"],
               [(esc(t), f"<code>{c}</code>", esc(m)) for t, c, m in GLOSSARY]),
         "<h2>Code key</h2>",
         table(["Code", "Meaning"], [(esc(c), esc(m)) for c, m in CODE_KEY])]

    o.append("<h2>Exercise model (proposed)</h2>"
             "<p>Body part → Movement → Exercise. Every exercise belongs to one movement. "
             "Every movement belongs to one body part. Muscles are described, never scored. "
             "No row has a published source yet. Unmarked exercises are not in the app yet.</p>")
    for part, label in BODY_PARTS:
        rows = []
        for m in (m for m in EXERCISE_MODEL if m["part"] == part):
            exs = "<br>".join(
                f"{esc(n)} <code>{i}</code>" + (f" <b>{TAG[w]}</b>" if TAG[w] else "")
                + (f" — {esc(note)}" if note else "") for i, n, w, note in m["exercises"])
            move = esc(m["movement"]) + (f" <span class='meta'>({m['side']})</span>"
                                         if m["side"] else "")
            rows.append((move, esc(m["muscles"]), exs))
        o.append(f"<p><b>{esc(label)}</b></p>")
        o.append(table(["Movement", "Main muscles", "Exercises: name, id"], rows))
    o.append("<p><b>Rules</b></p><ol>" + "".join(f"<li>{esc(r)}</li>" for r in EXERCISE_RULES)
             + "</ol>")
```

```
    notes = []
    for code, t in TECHNIQUES.items():
        n = NODE[code]
        o.append(f"<h2>{code} · {esc(n['name'])} <span class='meta'>(ages "
                 f"{esc(n.get('bands', ''))})</span></h2><p>{esc(t['purpose'])}</p>")
        o.append(table(["Step", "What it does", "Rule or source", "Status"],
                       [(f"{s['id']} {esc(s['step'])}", esc(s["does"]),
                         esc(", ".join(s["sources"]) or "Program rule"),
                         f"<span class='{s['status']}'>{s['status']}</span>")
                        for s in t["steps"]]))
        notes += [(s["id"], esc(s["note"])) for s in t["steps"] if s.get("note")]

    o.append("<h2>Deviations: where the code differs</h2>")
    o.append(table(["Id", "Agent", "Detail"],
                   [(r["id"], r["agent"], esc(r["text"])) for r in DEVIATIONS]))
    o.append("<h2>Owner decisions (not build instructions)</h2>")
    o.append(table(["Id", "Agent", "Status", "Decision"],
                   [(r["id"], r["agent"], esc(r["status"]), esc(r["text"]))
                    for r in OPEN_DECISIONS]))
    o.append("<h2>Appendix A: step notes</h2>")
    o.append(table(["Step", "Note"], notes))
    o.append("<h2>Appendix B: sources</h2>")
    o.append(table(["Id", "Citation", "Ages covered", "Used for", "How far checked"],
                   [(sid, esc(s["cite"]), esc(s["covers"]), esc(s["used"]), esc(s["checked"]))
                    for sid, s in SOURCES.items()]))
    o.append("</body></html>")
    return "".join(o)


HERE = pathlib.Path(__file__).resolve().parent
NAME = f"{DATE.replace('-', '')}_FIT-T_technique-tables_v{VERSION}"
OUT = HERE / "archive" / f"v{VERSION}_{DATE}" / f"{NAME}.html"
PDF = HERE / f"{NAME}.pdf"
CHROME = pathlib.Path(r"C:\Program Files\Google\Chrome\Application\chrome.exe")

if __name__ == "__main__":
    OUT.parent.mkdir(parents=True, exist_ok=True)
    OUT.write_text(build(), encoding="utf-8")
    print(f"{OUT.name} written to archive — {len(TECHNIQUES)} agent table(s), "
          f"{len(EXERCISE_MODEL)} movements, {len(SOURCES)} sources")
    if not CHROME.exists():
        raise SystemExit(f"Chrome not found at {CHROME}; {PDF.name} not printed")
    subprocess.run([str(CHROME), "--headless=new", "--disable-gpu", "--no-pdf-header-footer",
                    f"--print-to-pdf={PDF}", OUT.as_uri()], check=True, capture_output=True)
    print(f"{PDF.name} printed")
```